

TEMPER: Tensorized Efficient Manifold-constrained Parameterization for Expressive Residual Routing

Yuxuan Gu¹, Wuyang Zhou¹, Huijun Xing¹, Danilo Mandic¹

¹Department of Electrical and Electronic Engineering, Imperial College London
{yuxuan.gu21, wuyang.zhou19, h.xing23@imperial.ac.uk, d.mandic}@imperial.ac.uk

Abstract

Residual connections rely on a static residual pathway, and are essential for training deep neural networks. Hyperconnections (HC) increase the expressivity of residual routing by incorporating multiple residual streams and learning dynamic information flow, while manifold-constrained (mHC) variants stabilize training through doubly stochastic residual mixing. However, a generator-level bottleneck remains in existing methods: they use dense, unstructured generators for pre-branch aggregation, residual mixing, and post-branch redistribution, which results in parameter count growing rapidly with the number of streams. To address this issue, we propose **Tensorized Efficient Manifold-constrained Parameterization for Expressive Residual Routing (TEMPER)**, which represents these generators as multi-way tensors over the input-stream, feature, and output-stream modes, and parameterizes them using tensor networks. Such a structured low-rank formulation is shown to preserve token-dependent manifold-constrained routing interface while substantially reducing parameter growth. It also promotes interpretability and intuition, as: i) tensor ranks control the dimensionality of the learned routing subspace, with full ranks recovering dense routing; while ii) the generator approximation errors bound differences in routing logits and, consequently, in the routed-block outputs. Comprehensive experiments show that TEMPER matches or outperforms existing methods across language modeling and commonsense reasoning tasks, while requiring substantially fewer additional parameters. At eight residual streams, TEMPER achieves the best CORE score while using about 84% fewer additional parameters than mHC, thus showing a stronger performance-parameter efficiency trade-off.

1 Introduction

Residual connections (He et al. 2016) are central to the stability and scalability of modern Transformers (Vaswani et al. 2017; Brown et al. 2020). In a standard residual block, a module computes an update which is added back to the token’s current representation. This design is simple and effective, but it routes every token via the same fixed residual path.

Recently, hyperconnections (HC) (Zhu et al. 2025) have enriched this design by maintaining multiple residual streams and learning token-dependent routing among them, thus enabling more complex residual topologies and information flow patterns. Instead of applying an identical and fixed residual connection to all tokens, these methods allow each

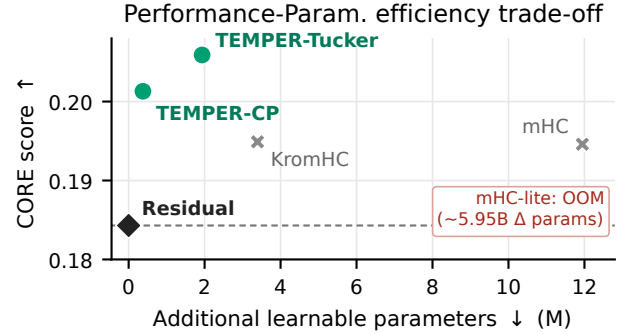


Figure 1: Comparison of CORE score and additional parameter cost at $n = 8$. Our TEMPER method provides the strongest trade-off among all hyper-connection variants.

token to adaptively aggregate, mix, and redistribute information across several streams. Manifold-constrained hyperconnections (mHC) (Xie et al. 2026) further improve the training stability of HC by using the iterative Sinkhorn-Knopp (SK) optimization (Sinkhorn 1964) to constrain the residual mixing matrix, $\mathbf{A}^{\text{res}}$, to be doubly stochastic. Subsequent variants have reparameterized this residual mixing step. Among these, the mHC-lite method (Yang and Gao 2026) achieves exact doubly stochastic residual mixing by constructing $\mathbf{A}^{\text{res}}$ as a convex combination of permutation matrices (Birkhoff 1946), while KromHC (Zhou et al. 2026) uses Kronecker products of smaller doubly stochastic matrices (Taranenko 2023), in order to avoid factorial growth of parameters with the number of streams incurred by mHC-lite. Figure 1 summarizes the performance-parameter trade-offs of the existing methods and the proposed TEMPER, and highlights the remaining parameter efficiency gap.

Although these HC variants improve the parameterization of the residual mixing matrix, the generator-level bottleneck still remains an open issue. For example, mHC uses unstructured dense generators to produce all three token-dependent routing projectors from stream-feature representations. As the number of streams increases, these generators become increasingly expensive in terms of both parameter count and computation, limiting the scalability of the expressive token-dependent routing. This raises the following question:

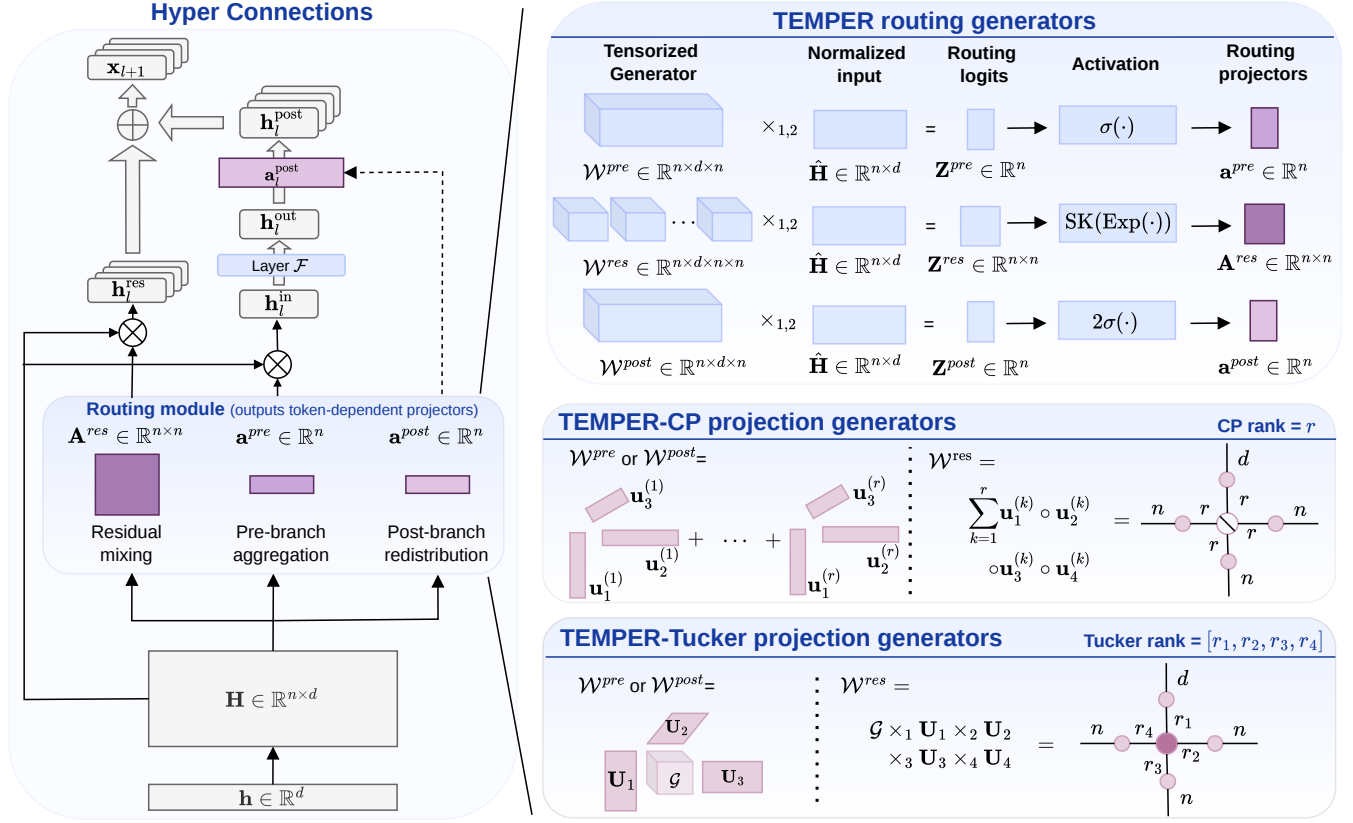


Figure 2: Overview of TEMPER architecture. TEMPER replaces unstructured dense hyper-connections routing generators with tensorized TEMPER-CP or TEMPER-Tucker generators, while preserving the same token-dependent routing interfaces.

- *Can we replace the dense unstructured routing generators in mHC with structured, parameter-efficient alternatives, while achieving on par or even better performance?*

To answer this question, we propose **Tensorized Efficient Manifold-constrained Parameterization for Expressive Residual Routing (TEMPER)**, which replaces the dense routing generators with parameter-efficient tensor-network parameterizations. By exploiting the inherently multi-way structure of mHC routing, TEMPER represents the generators for pre-branch aggregation, residual mixing, and post-branch redistribution as higher-order tensors over their input-stream, feature, and output-stream modes, and factorizes each independently using a low-rank tensor network.

TEMPER retains the original token-dependent routing interface, including doubly stochastic residual mixing, while inducing structured parameter sharing within each generator across stream–feature–output interactions. Its tensor ranks provide an explicit capacity control. Compact ranks restrict routing to low-dimensional learned subspaces, whereas sufficiently large ranks recover the dense parameterization. TEMPER thus reduces routing-generator parameter growth without altering the surrounding hyper-connection architecture.

Empirically, we instantiate TEMPER with Canonical Polyadic (CP) (Hitchcock 1927) and Tucker (Tucker 1966) parameterizations. Both outperform prior hyper-connection

baselines in downstream performance while using substantially fewer additional parameters. At eight residual streams, TEMPER-Tucker obtains the best CORE score (0.206 versus 0.195 for both mHC and KromHC) with 1.93M additional parameters, which is 84% fewer than 11.95M in mHC.

In summary, our contributions are as follows:

- We introduce **TEMPER**, a tensorized manifold-constrained hyper-connections method that replaces dense token-dependent routing generators with structured efficient tensor-network parameterizations, while preserving the original mHC routing interface.
- We analyze TEMPER’s scaling and expressivity. At fixed rank, routing-generator parameter grows linearly with the number of streams. Full-rank TEMPER-Tucker recovers dense routing, while generator approximation errors bound routing-logit and block-output deviations.
- We instantiate TEMPER with CP and Tucker parameterizations and evaluate them on language modeling and commonsense reasoning. TEMPER achieves a stronger performance–parameter efficiency trade-off than the existing residual and hyper-connection baselines.

2 Related Work

In a Transformer block $f(\cdot)$, standard residual connections (He et al. 2016) update every token’s latent representation

using the same pre-defined residual pathway

$$\mathbf{h}' = \mathbf{h} + f(\mathbf{h}), \quad (1)$$

where $\mathbf{h}$ is the hidden representation. While this design is computationally efficient and stable in optimization, it imposes the identical information flow pattern across all tokens.

2.1 Hyper-Connections Variants

Hyper-connections (HC) (Zhu et al. 2025) address this limitation by expanding the residual path to n streams and introducing token-specific routing over these streams. This increases the topological and representational complexity of the residual path, enabling richer and more diverse feature interactions across streams.

Specifically, at a block l , each token’s representation is repeated n times to obtain the expanded state matrix $^1 \mathbf{H}^{(l)} = [\mathbf{h}_1^{(l)\top}, \dots, \mathbf{h}_n^{(l)\top}]^\top \in \mathbb{R}^{n \times d}$. A routing module produces three token-dependent projection matrices $\mathbf{a}^{\text{pre}} \in \mathbb{R}^{1 \times n}$, $\mathbf{A}^{\text{res}} \in \mathbb{R}^{n \times n}$, and $\mathbf{a}^{\text{post}} \in \mathbb{R}^{1 \times n}$, with the resulting update

$$\mathbf{H}' = \mathbf{A}^{\text{res}} \mathbf{H} + (\mathbf{a}^{\text{post}})^T f(\mathbf{a}^{\text{pre}} \mathbf{H}). \quad (2)$$

Thus, the residual mixing is no longer globally fixed. Instead, it is selected dynamically for each token.

Manifold-Constrained Hyper-Connections (mHC) (Xie et al. 2026) constrains the residual mixing matrix, $\mathbf{A}^{\text{res}}$, to be doubly stochastic, since an unconstrained matrix can disrupt the identity-mapping behavior which is the key to training stability in standard residual networks. Let $\hat{\mathbf{h}} = \text{vec}(\text{RMSNorm}(\mathbf{H})) \in \mathbb{R}^{1 \times nd}$ be the normalized flattened state. Then, mHC generates the routing parameters through

$$\begin{aligned} \mathbf{a}^{\text{pre}} &= \sigma(\alpha^{\text{pre}} \hat{\mathbf{h}} \mathbf{W}^{\text{pre}} + \mathbf{b}^{\text{pre}}), \\ \mathbf{A}^{\text{res}} &= \text{SK}\left(\exp\left(\text{mat}\left(\alpha^{\text{res}} \hat{\mathbf{h}} \mathbf{W}^{\text{res}} + \mathbf{b}^{\text{res}}\right)\right)\right), \\ \mathbf{a}^{\text{post}} &= 2\sigma\left(\alpha^{\text{post}} \hat{\mathbf{h}} \mathbf{W}^{\text{post}} + \mathbf{b}^{\text{post}}\right), \end{aligned} \quad (3)$$

Here, $\mathbf{W}^{\text{pre}}, \mathbf{W}^{\text{post}} \in \mathbb{R}^{nd \times n}$, $\mathbf{b}^{\text{pre}}, \mathbf{b}^{\text{post}} \in \mathbb{R}^{1 \times n}$, $\mathbf{W}^{\text{res}} \in \mathbb{R}^{nd \times n^2}$, $\mathbf{b}^{\text{res}} \in \mathbb{R}^{1 \times n^2}$, $\alpha^{\text{pre}}, \alpha^{\text{post}}, \alpha^{\text{res}} \in \mathbb{R}$, and $\sigma(\cdot)$ denotes the Sigmoid($\cdot$) function. Moreover, $\text{SK}(\cdot)$ denotes the Sinkhorn–Knopp operator (Sinkhorn 1964), which iteratively normalizes $\mathbf{A}^{\text{res}}$ towards a doubly stochastic matrix. As a result, residual mixing becomes a non-expansive convex combination over streams, which improves training stability.

mHC-lite and KromHC. The mHC-lite method (Yang and Gao 2026) replaces iterative SK normalization with a Birkhoff–von Neumann mixture, while KromHC (Zhou et al. 2026) factorizes that mixture using smaller Kronecker factors. Both leave the pre- and post-routing generators unchanged. Their residual mixers can be summarized as

$$\begin{aligned} \mathbf{A}_{\text{lite}}^{\text{res}} &= \sum_{m=1}^{n!} \pi_m \mathbf{P}_m, \\ \mathbf{A}_{\text{Krom}}^{\text{res}} &= \bigotimes_{k=K}^1 \mathbf{U}^k, \quad \mathbf{U}^k = \sum_{m=1}^{i_k!} \pi_m^k \mathbf{P}_m^k, \end{aligned} \quad (4)$$

¹The superscript (l) is omitted for clarity from here onwards.

where $n = \prod_{k=1}^K i_k$, and π and π^k are token-dependent SoftMax weights over the corresponding permutation matrices. Both constructions are exactly doubly stochastic, but mHC-lite requires $\mathcal{O}(n!)$ mixture components, whereas KromHC reduces this cost via smaller Kronecker factors.

2.2 Tensor Methods in Large Language Models (LLMs)

Tensor decompositions, by their nature, improve parameter efficiency across LLM components. The Tensor Train decomposition (Oseledets 2011) compresses dense layers (Novikov et al. 2015), while TensorGPT (Xu, Xu, and Mandic 2023) extends this approach to token embeddings. Tensor-LLM (Gu et al. 2025) and Tucker Attention (Klein et al. 2026) apply Tucker-style factorizations to multi-head attention. In parameter-efficient fine-tuning, LoRETTA (Yang et al. 2024) uses Tensor Train adapters, whereas TeRA (Gu et al. 2026) enables high-rank adaptation with a Tucker-like network.

The performance of tensorized models depends critically on their ranks and network structures. Zhou et al. (2025) study tensor-network ranks as design parameters, while Zeng et al. (2024) and Iacovides et al. (2025) explore domain-aware tensor-network structure search with LLM assistance.

3 Preliminaries

Notation. Scalars, vectors, matrices, and tensors are denoted by x , $\mathbf{x}$, $\mathbf{X}$, and $\mathcal{X}$, respectively (Kolda and Bader 2009; Cichocki et al. 2015, 2016). We use $(\cdot)^\top$ for matrix transpose, $\mathcal{A}_{(i_1, \dots, i_N)}$ for the $(i_1, \dots, i_N)$ -th tensor entry, $\times_n$ for the mode- n product, $\circ$ for the outer product, and $\otimes$ for the Kronecker product. The operators $\text{vec}(\cdot)$ and $\text{mat}(\cdot)$ denote the vectorization and matricization, respectively.

Tensor Contraction. Contractions are used to sum over shared modes of compatible tensors. For $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ and $\mathbf{X} \in \mathbb{R}^{I_1 \times I_2}$, the contraction over the first two modes is

$$(\mathcal{A} \times_{1,2} \mathbf{X})_{i_3, \dots, i_N} = \sum_{u=1}^{I_1} \sum_{v=1}^{I_2} \mathcal{A}_{u,v,i_3, \dots, i_N} \mathbf{X}_{u,v}. \quad (5)$$

For $\mathcal{A}$ and $\mathcal{B}$ with mode a of $\mathcal{A}$ and mode b of $\mathcal{B}$ both of size R , tensor–tensor contraction

$$(\mathcal{A} \times_a^b \mathcal{B})_{i_1, \dots, i_{N-1}, j_1, \dots, j_{M-1}} = \sum_{t=1}^R \mathcal{A}_{i_1, \dots, i_{N-1}, t} \mathcal{B}_{t, j_1, \dots, j_{M-1}}, \quad (6)$$

with all other modes remaining uncontracted.

Unfolding. Unfolding, or matricization, reshapes a tensor into a matrix by mapping the indices of selected modes to the matrix rows and the indices of the remaining modes to the matrix columns. For $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the unfolding that groups the first two modes against the remaining modes is given by

$$\text{mat}(\mathcal{A}) \in \mathbb{R}^{I_1 I_2 \times \prod_{m=3}^N I_m}, \quad (7)$$

while the entries of the unfolded matrix are given by

$$\text{mat}(\mathcal{A})_{(i_1-1)I_2+i_2, 1+\sum_{m=3}^N (i_m-1) \prod_{\ell=m+1}^N I_\ell} = \mathcal{A}_{i_1, i_2, \dots, i_N}. \quad (8)$$

4 Methodology

Existing hyper-connection variants mainly differ in how they constrain or parameterize the residual mixing matrix, $\mathbf{A}^{\text{res}}$. Our focus is on re-designing the dense routing generators through the lens of tensor factorization. The overall architecture is illustrated in Figure 2.

4.1 A Tensor View of Dense Routing Generators

We now focus on the generators that produce the routing projectors. While prior methods differ in their construction of $\mathbf{A}^{\text{res}}$, they all produce $\mathbf{a}^{\text{pre}}$, $\mathbf{A}^{\text{res}}$, and $\mathbf{a}^{\text{post}}$ using dense maps. Our TEMPER replaces these dense generators with tensorized maps, while preserving the same routing interface.

Let $\hat{\mathbf{H}} = \text{RMSNorm}(\mathbf{H}) \in \mathbb{R}^{n \times d}$ denote the normalized stream state. Instead of flattening $\hat{\mathbf{H}}$ into a length- nd vector, we retain its stream and feature modes. The dense generators can then be viewed as the unfolding of tensors $\mathcal{W}^{\text{pre}} \in \mathbb{R}^{n \times d \times n}$, $\mathcal{W}^{\text{post}} \in \mathbb{R}^{n \times d \times n}$, and $\mathcal{W}^{\text{res}} \in \mathbb{R}^{n \times d \times n \times n}$, where the first two modes correspond to the input-streams and features, and the remaining modes index the routing output-streams dimensions. With this tensor view, dense mHC generators can be re-written as

$$\begin{aligned} \mathbf{a}^{\text{pre}} &= \sigma(\alpha^{\text{pre}} \mathcal{Z}^{\text{pre}} + \mathbf{b}^{\text{pre}}), \\ \mathbf{A}^{\text{res}} &= \text{SK}\left(\exp\left(\alpha^{\text{res}} \mathcal{Z}^{\text{res}} + \text{mat}(\mathbf{b}^{\text{res}})\right)\right), \\ \mathbf{a}^{\text{post}} &= 2\sigma(\alpha^{\text{post}} \mathcal{Z}^{\text{post}} + \mathbf{b}^{\text{post}}), \end{aligned} \quad (9)$$

where the three routing logits $\mathcal{Z}^{\text{pre}}$, $\mathcal{Z}^{\text{res}}$, $\mathcal{Z}^{\text{post}}$ are obtained by contracting the input-stream and feature modes as

$$\begin{aligned} \mathcal{Z}^{\text{pre}} &= \mathcal{W}^{\text{pre}} \times_{1,2} \hat{\mathbf{H}}, & \mathcal{Z}_i^{\text{pre}} &= \sum_{s=1}^n \sum_{c=1}^d \hat{H}_{s,c} \mathcal{W}_{s,c,i}^{\text{pre}}, \\ \mathcal{Z}^{\text{res}} &= \mathcal{W}^{\text{res}} \times_{1,2} \hat{\mathbf{H}}, & \mathcal{Z}_{i,j}^{\text{res}} &= \sum_{s=1}^n \sum_{c=1}^d \hat{H}_{s,c} \mathcal{W}_{s,c,i,j}^{\text{res}}, \\ \mathcal{Z}^{\text{post}} &= \mathcal{W}^{\text{post}} \times_{1,2} \hat{\mathbf{H}}, & \mathcal{Z}_i^{\text{post}} &= \sum_{s=1}^n \sum_{c=1}^d \hat{H}_{s,c} \mathcal{W}_{s,c,i}^{\text{post}}. \end{aligned} \quad (10)$$

Remark 1. The tensor view in Eq. (9) and (10) is equivalent to the original dense generators as matricizing each generator tensor recovers its corresponding dense weight matrix. TEMPER changes only how these tensors are parameterized.

Thus, dense routing learns independent parameters for each stream-feature-output interaction, with no explicit sharing across modes. This can be inefficient when routing decisions depend on reusable shared stream-feature-output patterns. TEMPER introduces such parameter sharing by factorizing the generator tensors with structured tensor networks.

4.2 Tensorizing the Routing Generators

The tensorization is achieved by keeping the contractions in Eq. (10) fixed and by replacing each dense generator tensor with a structured CP or Tucker tensor factorization.

TEMPER-CP. TEMPER-CP uses CP decomposition to factorize each generator tensor into rank-one outer products

$$\begin{aligned} \mathcal{W}^{\text{pre}} &= \sum_{q=1}^r \mathbf{u}_q^{\text{pre}} \circ \mathbf{c}_q^{\text{pre}} \circ \mathbf{v}_q^{\text{pre}}, \\ \mathcal{W}^{\text{res}} &= \sum_{q=1}^r \mathbf{u}_q^{\text{res}} \circ \mathbf{c}_q^{\text{res}} \circ \mathbf{v}_{1,q}^{\text{res}} \circ \mathbf{v}_{2,q}^{\text{res}}, \\ \mathcal{W}^{\text{post}} &= \sum_{q=1}^r \mathbf{u}_q^{\text{post}} \circ \mathbf{c}_q^{\text{post}} \circ \mathbf{v}_q^{\text{post}}. \end{aligned} \quad (11)$$

Here, $\mathbf{u}_q \in \mathbb{R}^n$ and $\mathbf{c}_q \in \mathbb{R}^d$ are factors for the input-stream and feature mode, respectively, while $\mathbf{v}_q \in \mathbb{R}^n$ is the output-stream mode factor. For $\mathcal{W}^{\text{res}}$, $\mathbf{v}_{1,q}, \mathbf{v}_{2,q} \in \mathbb{R}^n$ denote factors for its two output-stream modes. Within each generator, the component index, q , couples the mode-specific factors in each rank-one term, inducing structured parameter sharing across stream-feature-output interactions.

TEMPER-Tucker. TEMPER-Tucker employs Tucker decomposition to factorize each generator tensor using factor matrices and a small tensor core as

$$\begin{aligned} \mathcal{W}^{\text{pre}} &= \mathcal{G}^{\text{pre}} \times_1 \mathbf{U}^{\text{pre}} \times_2 \mathbf{C}^{\text{pre}} \times_3 \mathbf{V}^{\text{pre}}, \\ \mathcal{W}^{\text{res}} &= \mathcal{G}^{\text{res}} \times_1 \mathbf{U}^{\text{res}} \times_2 \mathbf{C}^{\text{res}} \times_3 \mathbf{V}_1^{\text{res}} \times_4 \mathbf{V}_2^{\text{res}}, \\ \mathcal{W}^{\text{post}} &= \mathcal{G}^{\text{post}} \times_1 \mathbf{U}^{\text{post}} \times_2 \mathbf{C}^{\text{post}} \times_3 \mathbf{V}^{\text{post}}. \end{aligned} \quad (12)$$

We use Tucker ranks (r_n, r_d, r_n) for the pre- and post-branch generators and (r_n, r_d, r_n, r_n) for the residual generator, with input- and output-stream factors $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{n \times r_n}$ and feature factors $\mathbf{C} \in \mathbb{R}^{d \times r_d}$. Given $\hat{\mathbf{H}} \in \mathbb{R}^{n \times d}$, the input is first projected into a compressed stream-feature representation:

$$\begin{aligned} \mathbf{T}^{\text{pre}} &= (\mathbf{U}^{\text{pre}})^\top \hat{\mathbf{H}} \mathbf{C}^{\text{pre}}, \\ \mathbf{T}^{\text{res}} &= (\mathbf{U}^{\text{res}})^\top \hat{\mathbf{H}} \mathbf{C}^{\text{res}}, \\ \mathbf{T}^{\text{post}} &= (\mathbf{U}^{\text{post}})^\top \hat{\mathbf{H}} \mathbf{C}^{\text{post}}. \end{aligned} \quad (13)$$

The routing logits are then obtained by contracting this representation with the core and the output modes:

$$\begin{aligned} \mathcal{Z}^{\text{pre}} &= (\mathcal{G}^{\text{pre}} \times_{1,2} \mathbf{T}^{\text{pre}}) \times_1 \mathbf{V}^{\text{pre}}, \\ \mathcal{Z}^{\text{res}} &= (\mathcal{G}^{\text{res}} \times_{1,2} \mathbf{T}^{\text{res}}) \times_1 \mathbf{V}_1^{\text{res}} \times_2 \mathbf{V}_2^{\text{res}}, \\ \mathcal{Z}^{\text{post}} &= (\mathcal{G}^{\text{post}} \times_{1,2} \mathbf{T}^{\text{post}}) \times_1 \mathbf{V}^{\text{post}}. \end{aligned} \quad (14)$$

TEMPER-Tucker therefore constrains routing to depend on a compressed stream-feature representation rather than on an unrestricted dense map.

Parameter Efficiency. Factorizing the routing generators substantially reduces the parameter cost of token-dependent routing. The pre-, residual-, and post-routing outputs lie in $\mathbb{R}^n$, $\mathbb{R}^{n \times n}$, and $\mathbb{R}^n$, respectively. Counting generator weights only, the parameter count for each parameterization is

$$\begin{aligned} P_{\text{mHC}} &= d(n^3 + 2n^2), \\ P_{\text{CP}} &= 3dr + 7nr, \\ P_{\text{Tucker}} &= 7nr_n + 3dr_d + 2r_n^2 r_d + r_n^3 r_d. \end{aligned} \quad (15)$$

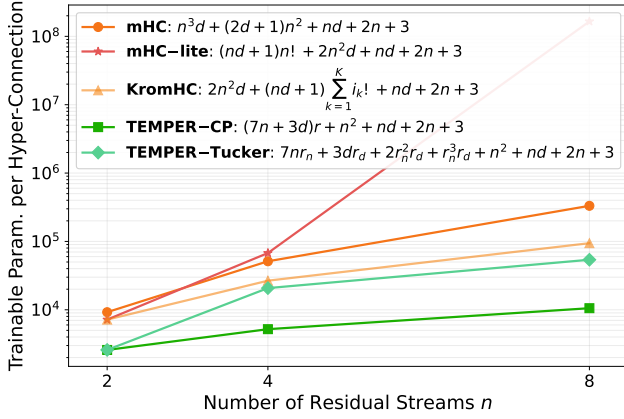


Figure 3: Trainable parameters per hyper-connection, including routing generators, biases, scalar gates, and RMSNorm. TEMPER scales more favorably with the number of streams.

Here, the rank r is shared across the three TEMPER-CP generators, while TEMPER-Tucker uses input- and output-stream rank, r_n , and feature rank, r_d . Thus, for fixed d and tensor ranks, dense mHC scales as $\mathcal{O}(n^3)$ in the number of streams, whereas both TEMPER parameterizations scale as $\mathcal{O}(n)$. Figure 3 reports the complete learnable parameter counts in hyper-connections, including biases, scalar gates, and RMSNorm scales.

Remark 2. Prior work improves parameter efficiency mainly by reparameterizing $\mathbf{A}^{\text{res}}$. To our knowledge, TEMPER is the first to jointly reparameterize the pre-, residual-, and post-routing generators, thus achieving greater parameter efficiency while matching or improving performance.

4.3 Inductive Bias and Approximation Guarantees

As TEMPER-Tucker routes through the three compressed representations in Eq. (13), its ranks restrict which stream-feature directions affect routing and control how closely the tensorized generators approximate their dense counterparts. We formalize both properties below. TEMPER-CP is a special case of TEMPER-Tucker with a superdiagonal core.

Theorem 1. (Rank and Null-space Dimension of the Tucker Compression) For any of the three generators in Eq. (12), assume that $\mathbf{U} \in \mathbb{R}^{n \times r_n}$ and $\mathbf{C} \in \mathbb{R}^{d \times r_d}$ have full column rank. Given the normalized input $\hat{\mathbf{H}} \in \mathbb{R}^{n \times d}$, define

$$L : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{r_n \times r_d}, \quad L(\hat{\mathbf{H}}) = \mathbf{U}^\top \hat{\mathbf{H}} \mathbf{C}. \quad (16)$$

Then, $\text{rank}(L) = r_n r_d$ and $\dim \mathcal{N}(L) = nd - r_n r_d$.

Theorem 1 shows that routing depends only on an $r_n r_d$ -dimensional learned subspace and is invariant to the remaining directions. Increasing ranks enlarges this subspace, whereas low ranks impose a stronger bottleneck that may suppress noisy or redundant directions. At full multilinear rank, TEMPER-Tucker recovers the dense mHC generator, so Tucker ranks interpolate between compact and dense routing.

Remark 3. Full-rank recovery is an expressivity guarantee rather than our design objective. When routing-relevant interactions lie in a low-dimensional structured subspace, compact ranks can provide sufficient routing capacity.

Lemma 1. (Generator-to-logit Error Bound). Let $\mathcal{W}$ and $\tilde{\mathcal{W}}$ denote any corresponding dense and tensorized routing generators. For any normalized stream state $\hat{\mathbf{H}}$, define $\mathcal{Z}_{\mathcal{W}}(\hat{\mathbf{H}}) = \mathcal{W} \times_{1,2} \hat{\mathbf{H}}$, and similarly for $\tilde{\mathcal{W}}$. Then, the corresponding routing logits satisfy

$$\left\| \mathcal{Z}_{\mathcal{W}}(\hat{\mathbf{H}}) - \mathcal{Z}_{\tilde{\mathcal{W}}}(\hat{\mathbf{H}}) \right\|_F \leq \|\hat{\mathbf{H}}\|_F \left\| \mathcal{W} - \tilde{\mathcal{W}} \right\|_F. \quad (17)$$

Theorem 2. (End-to-End Routed-Block Approximation Bound) Let $\hat{\mathbf{H}} = \text{RMSNorm}(\mathbf{H})$ be the normalized stream state, and $\mathbf{H}'$ and $\tilde{\mathbf{H}}'$ the outputs of the dense and tensorized routed blocks. Let f be the shared residual transformation. Suppose that f is L_f -Lipschitz and satisfies $\|f(\mathbf{x})\|_2 \leq M_f$ for all relevant inputs, and that $\Phi(\mathbf{Z}) = \text{SK}(\exp(\mathbf{Z}))$ is L_{SK} -Lipschitz on the residual-logit domain. Then,

$$\begin{aligned} \left\| \mathbf{H}' - \tilde{\mathbf{H}}' \right\|_F &\leq |\alpha^{\text{res}}| L_{\text{SK}} \|\hat{\mathbf{H}}\|_F \|\mathbf{H}\|_F \left\| \mathcal{W}^{\text{res}} - \tilde{\mathcal{W}}^{\text{res}} \right\|_F \\ &\quad + \frac{|\alpha^{\text{post}}|}{2} M_f \|\hat{\mathbf{H}}\|_F \left\| \mathcal{W}^{\text{post}} - \tilde{\mathcal{W}}^{\text{post}} \right\|_F \\ &\quad + \frac{|\alpha^{\text{pre}}|}{2} \sqrt{n} L_f \|\hat{\mathbf{H}}\|_F \|\mathbf{H}\|_F \left\| \mathcal{W}^{\text{pre}} - \tilde{\mathcal{W}}^{\text{pre}} \right\|_F. \end{aligned} \quad (18)$$

Lemma 1 quantifies how tensor approximation error propagates to the routing logits. It shows that the discrepancy between dense and tensorized routing logits is bounded by the generator error $\left\| \mathcal{W} - \tilde{\mathcal{W}} \right\|_F$ scaled by the normalized input magnitude $\|\hat{\mathbf{H}}\|_F$. Since RMS normalization keeps this magnitude controlled, the resulting logit discrepancy is governed primarily by the quality of tensor approximation.

Theorem 2 extends this result to the complete routed block. Under the regularity assumptions, the block-output difference is bounded by the approximation errors of the pre-, residual-, and post-routing generators. Together, these results show how approximation errors are controlled as the routing parameterization moves from compact low-rank structure towards the dense unrestricted model. These guarantees consider expressivity and approximation, rather than generalization, and Section 5.4 empirically examines why increasing the rank might not necessarily improve performance. The proofs of theorems and lemma are given in the Supplement Section 1.

5 Experiments

To evaluate the performance of TEMPER, we used the nanochat (Karpathy 2025) backbone and apply different hyper-connections modules in different methods. The base model had $D = 12$ Transformer blocks, hidden dimension $d = 768$, 6 attention heads, context length 2048. We replaced residual branches in both attention and feed-forward network in every block, yielding 24 residual modules per model. The pretraining used ClimBMix-400B dataset (Diao et al. 2025) with a 32,768-token BPE tokenizer. $4 \times$ NVIDIA A100 (80GB) GPUs are used for the experiments. All hyper-parameters are provided in Supplement Section 4.

Table 1: Additional parameters, training loss, validation BPB, and CORE score for hyper-connections variants with $D = 12$ blocks and two residual connections per block. The $n = 4$ and $n = 8$ settings are respectively trained for 6,000 and 7,000 steps.

Method	$n = 4$ (6000 steps)				$n = 8$ (7000 steps)			
	Δ Params (K) $\downarrow$	Train Loss $\downarrow$	Val BPB $\downarrow$	CORE Score $\uparrow$	Δ Params (K) $\downarrow$	Train Loss $\downarrow$	Val BPB $\downarrow$	CORE Score $\uparrow$
Residual	0	2.705	0.816	0.182	0	2.645	<u>0.811</u>	0.184
mHC	1,844	2.714	0.819	0.177	11,946	2.627	<u>0.811</u>	0.195
mHC-lite	2,434	2.724	<u>0.812</u>	0.183	5,948,900	—	—	—
KromHC	959	2.691	<u>0.812</u>	0.186	3,392	2.612	0.807	0.195
TEMPER-CP (Ours)	186	<u>2.692</u>	<u>0.812</u>	<u>0.189</u>	376	<u>2.613</u>	0.807	<u>0.201</u>
TEMPER-Tucker (Ours)	<u>744</u>	2.720	0.811	0.192	<u>1,934</u>	<u>2.613</u>	0.807	0.206

5.1 Initialization

We followed KromHC (Zhou et al. 2026) for routing-bias and scalar-gate initialization, yielding stream-biased pre- and post-routing and near-identity residual mixing. Beyond this, all TEMPER-CP factor entries were initialized independently from $\mathcal{N}(0, r^{-1/2})$, where r is the CP rank. For TEMPER-Tucker, all factor and core entries are initialized independently from $\mathcal{N}(0, r_{\max}^{-1/2})$, where $r_{\max}$ is the maximum mode rank. This rank-scaled initialization keeps factor magnitudes controlled and improves optimization stability.

5.2 Evaluation Metrics

Following KromHC (Zhou et al. 2026), to measure efficiency and language-modeling quality, we report additional learnable parameters, training cross-entropy (CE) loss, and validation bits-per-byte (BPB). Additionally, we report the CORE score (Li et al. 2024), the mean-centered accuracy over 22 tasks (20 shown here), as a summary of downstream performance (See Supplement Section 3). For each task, we report task accuracy (%). Language-modeling tasks were scored by exact continuation match, while multiple-choice tasks select the option with the lowest continuation loss.

5.3 Downstream Performance

Tables 1, 2, and 3 report downstream performance for $n = 4$ and $n = 8$ hyper-connection streams. Across both settings, TEMPER provides the strongest performance–parameter efficiency trade-off among the hyper-connection baselines, matching or improving the downstream performance with substantially fewer additional parameters. mHC-lite results at $n = 8$ are unavailable because its $\mathcal{O}(n!)$ residual-routing parameter growth led to an out-of-memory (OOM) error.

As shown in Table 1, TEMPER-Tucker achieves the highest CORE score at both $n = 4$ and $n = 8$, improving over KromHC from 0.186 to 0.192 and from 0.195 to 0.206, respectively. At $n = 8$, it uses 1.93M additional parameters, compared with 3.39M for KromHC and 11.95M for mHC.

TEMPER-Tucker also achieves the highest average accuracy on the commonsense and reasoning benchmark in Table 2, reaching 50.0% and 51.3%, respectively. At $n = 8$, it outperformed the strongest existing baseline by 1.5%. For the language-modeling and BBH-oriented suite in Table 3, TEMPER-CP achieves the highest average accuracy in both stream counts, with 27.1% for $n = 4$ and 27.3% for $n = 8$.

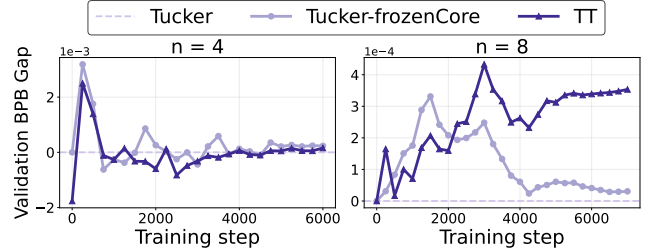


Figure 4: Smoothed validation BPB gaps relative to TEMPER-Tucker for the frozen-core and TT variant at $n = 4$ and $n = 8$. Positive values indicate worse BPB.

These results suggest an expressivity–regularization trade-off. TEMPER-Tucker’s learned core captures richer cross-mode interactions, which may benefit tasks requiring diverse routing patterns, such as commonsense reasoning. In comparison, TEMPER-CP imposes a stronger separable prior with greater parameter sharing, which may improve generalization on language-modeling and BBH-oriented tasks.

5.4 Ablation Studies

Frozen Tucker Core Eq. (14) separates Tucker routing into two stages. First, factor matrices construct a compressed input-stream-feature and output-stream representation, and then the core tensor models interactions among the compressed coordinates. The frozen-core variant isolates these roles by keeping the factor matrices trainable but fixing the core at initialization. As shown in Figure 4, the frozen-core model has consistently worse validation BPB than the fully trainable Tucker model. Thus, learning the projection subspaces alone is insufficient. Adapting the interactions among those subspaces also contributes to the routing quality.

Effect of Tensor Network Topology Beyond the CP and Tucker decompositions, we evaluated a Tensor Train (TT) variant (See Supplement Section 2), given by

$$\mathcal{W} = \mathcal{G}_1 \times_3^1 \mathcal{G}_2 \times_4^1 \mathcal{G}_3 \times_5^1 \cdots \times_{m+1}^1 \mathcal{G}_m, \quad (19)$$

where $m = 3$ for the pre- and post-branch generators and $m = 4$ for the residual generator. TT represents each generator as a chain of low-rank cores, rather than Tucker’s single global core. Figure 4 shows that TT has worse validation BPB than Tucker at both $n = 4$ and $n = 8$, suggesting that a global core better suits this routing setting.

Table 2: Commonsense reasoning results for two stream counts. Best results are bolded. Second best results are underlined.

n	Method	Avg	HS	HS-ZS	PIQA	ARC-E	ARC-C	COPA	CSQA	OBQA	WG	WGrande	BoolQ
4	Residual (6000 steps)	49.0	41.8	39.4	71.0	61.8	<u>33.0</u>	65.0	25.6	35.0	<u>61.9</u>	49.2	55.4
	mHC	48.3	39.4	38.0	70.4	60.4	30.8	59.0	28.8	34.2	60.1	52.8	57.2
	mHC-lite	48.9	42.6	<u>40.6</u>	71.0	60.2	33.2	60.0	22.4	33.0	61.5	56.0	57.4
	KromHC	<u>49.1</u>	41.0	<u>40.6</u>	70.8	60.8	32.8	60.0	29.8	33.2	58.2	<u>53.0</u>	60.0
	TEMPER-CP (Ours)	48.9	39.2	39.2	70.6	59.8	31.6	61.0	<u>34.4</u>	<u>32.4</u>	62.3	52.4	55.2
	TEMPER-Tucker (Ours)	50.0	41.0	41.6	68.2	60.2	30.8	<u>62.0</u>	39.6	<u>34.8</u>	60.1	52.2	<u>59.8</u>
8	Residual (7000 steps)	48.6	40.0	41.4	71.6	<u>63.2</u>	<u>32.8</u>	66.0	22.8	33.8	60.4	51.8	51.0
	mHC	49.5	40.6	<u>41.2</u>	72.2	61.6	32.0	64.0	<u>31.6</u>	33.8	<u>61.9</u>	50.4	55.4
	mHC-lite	—	—	—	—	—	—	—	—	—	—	—	—
	KromHC	49.8	40.6	40.6	72.4	62.6	33.2	<u>65.0</u>	29.0	33.8	63.0	53.0	54.2
	TEMPER-CP (Ours)	<u>50.1</u>	<u>41.6</u>	41.0	73.0	63.4	32.2	63.0	29.8	<u>34.2</u>	60.8	54.4	<u>57.6</u>
	TEMPER-Tucker (Ours)	51.3	42.0	40.8	<u>72.6</u>	62.8	32.6	<u>65.0</u>	42.2	35.6	58.2	<u>54.2</u>	58.0

Table 3: Language modeling results for the two stream counts. Best results are bolded. Second best results are underlined.

n	Method	Avg	Lamb	SQuAD	CoQA	BBH-QA	BBH-CS	BBH-Op	BBH-Dyck	LSAT	LangID
4	Residual (6000 steps)	25.2	<u>35.2</u>	25.2	19.4	36.4	<u>45.0</u>	11.0	5.0	21.7	28.2
	mHC	25.0	<u>33.6</u>	22.8	20.6	37.0	<u>40.2</u>	15.2	5.2	22.6	28.0
	mHC-lite	25.3	33.2	25.8	<u>22.0</u>	34.6	<u>45.0</u>	12.4	5.0	22.2	27.2
	KromHC	25.7	35.4	25.0	22.4	36.0	40.4	15.2	2.8	<u>25.2</u>	<u>29.0</u>
	TEMPER-CP (Ours)	27.1	34.8	27.2	22.4	38.8	45.4	<u>13.8</u>	<u>7.6</u>	26.5	27.2
	TEMPER-Tucker (Ours)	<u>25.9</u>	33.4	<u>26.0</u>	21.8	<u>37.4</u>	44.4	10.5	8.6	21.7	29.4
8	Residual (7000 steps)	26.9	34.2	25.8	23.0	39.6	46.2	13.8	9.6	23.9	25.6
	mHC	<u>27.1</u>	32.8	30.6	<u>22.4</u>	<u>38.0</u>	<u>45.6</u>	<u>13.8</u>	<u>9.2</u>	27.4	24.4
	mHC-lite	—	—	—	—	—	—	—	—	—	—
	KromHC	26.7	35.2	<u>31.8</u>	20.8	33.6	42.4	<u>13.8</u>	<u>9.2</u>	<u>26.1</u>	<u>27.4</u>
	TEMPER-CP (Ours)	27.3	<u>35.0</u>	34.4	22.2	37.8	42.2	12.9	7.4	25.7	28.2
	TEMPER-Tucker (Ours)	26.4	33.8	31.2	20.8	33.4	43.4	14.3	9.0	25.2	26.4

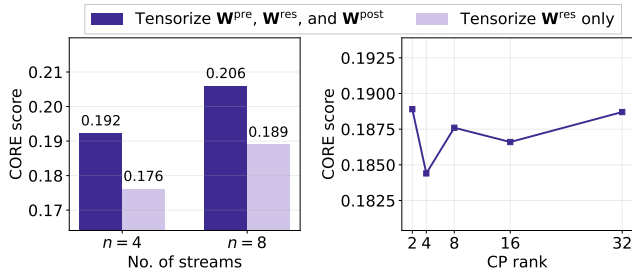


Figure 5: Ablations on TEMPER routing design. **Left:** CORE score comparison at the final step for full TEMPER-Tucker against a residual-only variant that tensorizes $\mathbf{W}^{\text{res}}$ only. **Right:** CORE score at final step for TEMPER-CP with ranks $r \in \{2, 4, 8, 16, 32\}$ at $n=4$.

Tensorizing only $\mathbf{W}^{\text{res}}$ Theorem 2 decomposes the block-output discrepancy into contributions from the three routing generators. To isolate the effect of the residual generator, we tensorized only $\mathbf{W}^{\text{res}}$, while keeping $\mathbf{W}^{\text{pre}}$ and $\mathbf{W}^{\text{post}}$ dense. As shown in Figure 5 (left), this variant underperformed full TEMPER-Tucker, whose CORE score increased from 0.176 to 0.192 at $n=4$, and from 0.189 to 0.206 at $n=8$. These results show that tensorizing the residual generator, $\mathbf{W}^{\text{res}}$,

is beneficial, while jointly tensorizing all three generators yields the best performance.

Tensor ranks Theorem 1 shows that tensor rank controls the dimension of the learned stream-feature subspace available for routing. Figure 5 (right) studies the effect of CP rank at $n=4$ under a fixed 6000-step budget. The final CORE score varied only from 0.184 to 0.189, with the best results at $r=2$. This weak performance sensitivity to rank suggests that the dense generator is over-parameterized for routing in this setting. A small number of shared low-rank components provides sufficient expressivity, while the resulting structural constraint may also act as an implicit regularizer.

6 Conclusion

We have introduced TEMPER, which jointly tensorizes pre-aggregation, residual-mixing, and post-redistribution routing generators in hyper-connections, reducing parameter growth while retaining effective routing capacity. Our theoretical analysis characterizes routing capacity through tensor ranks and bounds how generator approximation errors propagate to routing logits and block outputs. Across language modeling and reasoning benchmarks, TEMPER has matched or outperformed prior hyper-connection methods, while requiring substantially fewer additional parameters, providing a scalable approach to expressive hyper-connections in LLMs.

References

- Birkhoff, G. 1946. Tres Observaciones Sobre el Algebra Lineal. *Rev. Univ. Nac. Tucumán Ser. A*, 5: 147–151.
- Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; and Lin, H., eds., *Advances in Neural Information Processing Systems*, volume 33, 1877–1901. Curran Associates, Inc.
- Cichocki, A.; Lee, N.; Oseledets, I.; Phan, A.-H.; Zhao, Q.; and Mandic, D. P. 2016. Tensor Networks for Dimensionality Reduction and Large-scale Optimization Part 1 Low-rank Tensor Decompositions. *Foundations and Trends in Machine Learning*, 9(4-5): 249–429.
- Cichocki, A.; Mandic, D.; De Lathauwer, L.; Zhou, G.; Zhao, Q.; Caiafa, C.; and Phan, H. A. 2015. Tensor Decompositions for Signal Processing Applications: From Two-way to Multi-way Component Analysis. *IEEE signal processing magazine*, 32(2): 145–163.
- Diao, S.; Yang, Y.; Fu, Y.; Dong, X.; Su, D.; Kliegl, M.; Chen, Z.; Belcak, P.; Suhara, Y.; Yin, H.; Patwary, M.; Lin, C.; Kautz, J.; and Molchanov, P. 2025. CLIMB: CLustering-based Iterative Data Mixture Bootstrapping for Language Model Pre-training. *arXiv preprint*.
- Gu, Y.; Zhou, W.; Iacovides, G.; and Mandic, D. 2025. TensorLLM: Tensorising Multi-Head Attention for Enhanced Reasoning and Compression in LLMs. In *2025 International Joint Conference on Neural Networks (IJCNN)*, 1–8. IEEE.
- Gu, Y.; Zhou, W.; Iacovides, G.; and Mandic, D. 2026. TeRA: Vector-based Random Tensor Network for High-rank Adaptation of Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2314–2329.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.
- Hitchcock, F. L. 1927. The Expression of a Tensor or a Polyadic as a Sum of Products. *Journal of Mathematics and Physics*, 6(1-4): 164–189.
- Iacovides, G.; Zhou, W.; Li, C.; Zhao, Q.; and Mandic, D. 2025. Domain-aware Tensor Network Structure Search. *arXiv preprint arXiv:2505.23537*.
- Karpathy, A. 2025. nanochat: The best ChatGPT that \$100 can buy.
- Klein, T.; Kusch, J.; Sager, S.; Schnake, S.; and Schotthöfer, S. 2026. Tucker Attention: A Generalization of Approximate Attention Mechanisms. *arXiv preprint arXiv:2603.30033*.
- Kolda, T. G.; and Bader, B. W. 2009. Tensor Decompositions and Applications. *SIAM Review*, 51(3): 455–500.
- Li, J.; Fang, A.; Smyrnis, G.; Ivgi, M.; Jordan, M.; Gadre, S.; Bansal, H.; Guha, E.; Keh, S.; Arora, K.; Garg, S.; Xin, R.; Muennighoff, N.; Heckel, R.; Mercat, J.; Chen, M.; Gururangan, S.; Wortsman, M.; Albalak, A.; Bitton, Y.; Nezhurina, M.; Abbas, A.; Hsieh, C.-Y.; Ghosh, D.; Gardner, J.; Kilian, M.; Zhang, H.; Shao, R.; Pratt, S.; Sanyal, S.; Ilharco, G.; Daras, G.; Marathe, K.; Gokaslan, A.; Zhang, J.; Chandu, K.; Nguyen, T.; Vasiljevic, I.; Kakade, S.; Song, S.; Sanghavi, S.; Faghri, F.; Oh, S.; Zettlemoyer, L.; Lo, K.; El-Nouby, A.; Pouransari, H.; Toshev, A.; Wang, S.; Groeneveld, D.; Soldaini, L.; Koh, P. W.; Jitsev, J.; Kollar, T.; Dimakis, A. G.; Carmon, Y.; Dave, A.; Schmidt, L.; and Shankar, V. 2024. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems*, volume 37, 14200–14282. Curran Associates, Inc.
- Novikov, A.; Podoprikin, D.; Osokin, A.; and Vetrov, D. P. 2015. Tensorizing Neural Networks. *Advances in neural information processing systems*, 28.
- Oseledets, I. V. 2011. Tensor-Train Decomposition. *SIAM Journal on Scientific Computing*, 33(5): 2295–2317.
- Sinkhorn, R. 1964. A Relationship Between Arbitrary Positive Matrices and Doubly Stochastic Matrices. *The Annals of Mathematical Statistics*, 35(2): 876–879.
- Taranenko, A. A. 2023. Products of Multidimensional Matrices, Stochastic Matrices, and Permanents. *arXiv preprint arXiv:2303.17278*.
- Tucker, L. R. 1966. Some Mathematical Notes on Three-mode Factor Analysis. *Psychometrika*, 31(3): 279–311.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Xie, Z.; Wei, Y.; Cao, H.; Zhao, C.; Deng, C.; Li, J.; Dai, D.; Gao, H.; Xu, M.; Yu, K.; Zhao, L.; Zhou, S.; Xu, Z.; Zhang, Z.; Zeng, W.; Hu, S.; Wang, Y.; Yuan, J.; Wang, L.; and Liang, W. 2026. mHC: Manifold-Constrained Hyper-Connections. In *Forty-third International Conference on Machine Learning*.
- Xu, M.; Xu, Y. L.; and Mandic, D. P. 2023. Tensorgpt: Efficient Compression of Large Language Models based on Tensor-Train Decomposition. *arXiv preprint arXiv:2307.00526*.
- Yang, Y.; and Gao, J. 2026. mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations. *arXiv preprint arXiv:2601.05732*.
- Yang, Y.; Zhou, J.; Wong, N.; and Zhang, Z. 2024. LoRETTA: Low-Rank Economic Tensor-Train Adaptation for Ultra-Low-Parameter Fine-Tuning of Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics.
- Zeng, J.; Li, C.; Sun, Z.; Zhao, Q.; and Zhou, G. 2024. TNGPS: Discovering Unknown Tensor Network Structure

Search Algorithms via Large Language Models (LLMs). *arXiv preprint arXiv:2402.02456*.

Zhou, W.; Gu, Y.; Iacovides, G.; and Mandic, D. 2026. KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices. In *Forty-third International Conference on Machine Learning*.

Zhou, W.; Iacovides, G.; Konstantinidis, K.; Kisil, I.; and Mandic, D. 2025. Understanding the Rank of Tensor Networks via an Intuitive Example-Driven Approach. *arXiv preprint arXiv:2507.10170*.

Zhu, D.; Huang, H.; Huang, Z.; Zeng, Y.; Mao, Y.; Wu, B.; Min, Q.; and Zhou, X. 2025. Hyper-Connections. In *International Conference on Learning Representations*, volume 2025, 97183–97219.